PostgreSQL Database Backup, Disaster Recovery, and Streaming Replication

Author: Silvio Ukoth
Database: bootcamp
Technology: PostgreSQL
Operating System: Linux / Ubuntu

---

1. Project Overview

This project demonstrates practical PostgreSQL database administration, focusing on database backup, recovery, WAL archiving, point-in-time recovery (PITR), and streaming replication.

The main objective is to protect database information against accidental deletion or system failure and ensure database availability through replication.

2. Step 1: Take and Verify a Logical Backup

Objective

Create a logical backup of the `bootcamp` database and verify that it can be restored successfully.

2.1 Create a backup directory

mkdir -p ~/backups

2.2 Create the database backup

pg_dump -Fc -f ~/backups/bootcamp.dump bootcamp

2.3 Verify the backup file

ls -lh ~/backups/bootcamp.dump

2.4 Inspect the backup contents

pg_restore --list ~/backups/bootcamp.dump | head

2.5 Create a test database

createdb bootcamp_check

2.6 Restore the backup

pg_restore -d bootcamp_check ~/backups/bootcamp.dump

2.7 Verify the restored database

psql -d bootcamp_check -c "\dt"

Expected Results

- The `bootcamp.dump` file should exist.
- The backup archive should contain database objects.
- The `bootcamp_check` database should be created.
- The restored database should contain the original tables.

---

3. Step 2: Enable WAL Archiving

Objective

Configure PostgreSQL to archive Write-Ahead Log (WAL) files and create a physical base backup.

WAL files record database changes and allow PostgreSQL to recover transactions after a failure.

3.1 Create the WAL archive directory

sudo mkdir -p /var/lib/postgresql/wal_archive
sudo chown postgres:postgres /var/lib/postgresql/wal_archive
sudo chmod 700 /var/lib/postgresql/wal_archive

3.2 Find the PostgreSQL configuration file

sudo -u postgres psql -c "SHOW config_file;"

3.3 Edit postgresql.conf

Open the configuration file using its actual path:

sudo nano /etc/postgresql/16/main/postgresql.conf

Configure the following settings:

wal_level = replica

archive_mode = on

archive_command = 'test ! -f /var/lib/postgresql/wal_archive/%f && cp %p /var/lib/postgresql/wal_archive/%f'

max_wal_senders = 10

max_replication_slots = 10

Note: Replace the example PostgreSQL version and configuration path with the actual values on your computer.

3.4 Restart PostgreSQL

sudo systemctl restart postgresql

Check the service:

sudo systemctl status postgresql

3.5 Verify WAL configuration

sudo -u postgres psql -d bootcamp

Execute:

```
SHOW wal_level;
SHOW archive_mode;
SHOW archive_command;
```

3.6 Generate a WAL switch

SELECT pg_switch_wal();

Exit PostgreSQL:

\q

3.7 Check archived WAL files

sudo ls -lh /var/lib/postgresql/wal_archive/

3.8 Check archiver statistics

sudo -u postgres psql -d bootcamp

Execute:

```
SELECT
   archived_count,
   last_archived_wal,
   failed_count,
   last_failed_wal,
   last_failed_time,
```

```
    last_archived_time
FROM pg_stat_archiver;
```

Successful archiving should increase `archived_count`, update `last_archived_wal`, and update `last_archived_time`.

Ideally, `failed_count` should remain zero.

3.9 Create a physical base backup

Create the destination directory:

```
mkdir -p ~/backups/base
```

Execute:

```
pg_basebackup -D ~/backups/base -Fp -Xs -P
```

Explanation:

- `-D`: Specifies the backup destination.
- `-Fp`: Creates a plain directory-format backup.
- `-Xs`: Streams WAL during backup.
- `-P`: Displays backup progress.

Verify the backup:

```
ls -lh ~/backups/base
```

---

4. Step 3: Simulate a Disaster and Recover

Objective

Simulate accidental deletion of student records and recover the database to a point before the deletion using Point-in-Time Recovery (PITR).

Important: Perform this exercise only after verifying the base backup and WAL archives. Record the actual timestamp of your operation.

4.1 Connect to the bootcamp database

```
sudo -u postgres psql -d bootcamp
```

4.2 Check the existing student records

```
SELECT COUNT(*) FROM students;
```

Record the number of existing students.

4.3 Record the current time

SELECT now();

Save the timestamp before deleting the records.

4.4 Simulate accidental deletion

DELETE FROM students;

Verify the deletion:

SELECT COUNT(*) FROM students;

The result should be zero if the table contained records before deletion.

Exit PostgreSQL:

\q

4.5 Stop PostgreSQL

sudo systemctl stop postgresql

4.6 Identify the PostgreSQL data directory

pg_lsclusters

On a typical Ubuntu installation, the directory may be:

/var/lib/postgresql/16/main

Use your actual data directory.

4.7 Preserve the existing data directory

sudo mv /var/lib/postgresql/16/main \
/var/lib/postgresql/16/main_before_recovery

4.8 Restore the physical base backup

sudo cp -a ~/backups/base \
/var/lib/postgresql/16/main

Set ownership:

sudo chown -R postgres:postgres \
/var/lib/postgresql/16/main

4.9 Configure point-in-time recovery

Edit the restored configuration:

sudo nano /var/lib/postgresql/16/main/postgresql.conf

Add:

restore_command = 'cp /var/lib/postgresql/wal_archive/%f %p'

recovery_target_time = 'YOUR_RECORDED_TIMESTAMP'

recovery_target_action = 'pause'

Replace `YOUR_RECORDED_TIMESTAMP` with the actual recovery target.

The recovery target must be after the base backup and before the accidental deletion transaction commits.

4.10 Create the recovery signal file

sudo -u postgres touch \
/var/lib/postgresql/16/main/recovery.signal

4.11 Start PostgreSQL

sudo systemctl start postgresql

Check recovery logs:

sudo journalctl -u postgresql -n 50 --no-pager

4.12 Verify recovered student records

Connect:

sudo -u postgres psql -d bootcamp

Execute:

SELECT COUNT(*) FROM students;

The recovered count should match the records present at the selected recovery point.

After confirming recovery, promote the database to normal operation if it is paused:

SELECT pg_promote();

---

5. Step 4: Set Up a Streaming Standby

Objective

Create a standby PostgreSQL server that continuously receives changes from the primary server.

For this practical exercise, the primary and standby will run on the same computer using different ports.

- Primary server: 127.0.0.1:5432
- Standby server: 127.0.0.1:5433

5.1 Create a replication role

Connect to the primary:

sudo -u postgres psql -d bootcamp

Execute:

CREATE ROLE replicator
WITH REPLICATION LOGIN PASSWORD 'reppass';

Verify:

\du

5.2 Configure pg_hba.conf

Find the authentication configuration file:

sudo -u postgres psql -c "SHOW hba_file;"

Open the returned file:

sudo nano /etc/postgresql/16/main/pg_hba.conf

Add:

host replication replicator 127.0.0.1/32 scram-sha-256

5.3 Configure the primary server

Edit `postgresql.conf`:

listen_addresses = 'localhost'

wal_level = replica

max_wal_senders = 10

max_replication_slots = 10

Restart PostgreSQL:

sudo systemctl restart postgresql

5.4 Create the standby directory

sudo mkdir -p /var/lib/postgresql/standby

sudo chown postgres:postgres /var/lib/postgresql/standby

5.5 Build the standby using pg_basebackup

```
sudo -u postgres pg_basebackup \
-h 127.0.0.1 \
-p 5432 \
-U replicator \
-D /var/lib/postgresql/standby \
-R -P
```

Enter the replication password when requested.

The `-R` option automatically creates the standby signal file and replication connection configuration.

5.6 Configure the standby port

Edit:

sudo nano /var/lib/postgresql/standby/postgresql.conf

Add:

port = 5433

listen_addresses = 'localhost'

5.7 Start the standby server

```
sudo -u postgres pg_ctl \
-D /var/lib/postgresql/standby \
-o "-p 5433" \
-l /var/lib/postgresql/standby.log start
```

Check its status:

```
sudo -u postgres pg_ctl \
-D /var/lib/postgresql/standby status
```

5.8 Verify standby recovery mode

sudo -u postgres psql -p 5433 -d bootcamp

Execute:

SELECT pg_is_in_recovery();

Expected result:

t

This confirms that the server is operating as a standby.

---

6. Step 5: Watch Replication Health

Objective

Monitor the streaming replication connection, replication state, and lag between the primary and standby servers.

6.1 Connect to the primary server

sudo -u postgres psql -p 5432 -d bootcamp

6.2 Check replication status

Execute:

```
SELECT
  application_name,
  state,
  sent_lsn,
  write_lsn,
  flush_lsn,
  replay_lsn,
  pg_wal_lsn_diff(sent_lsn, replay_lsn) AS lag_bytes
FROM pg_stat_replication;
```

6.3 Understand the results

- `application_name`: Identifies the standby connection.
- `state`: Shows the replication state.
- `sent_lsn`: Shows the latest WAL location sent.
- `write_lsn`: Shows the latest WAL location written.
- `flush_lsn`: Shows the latest WAL location flushed.
- `replay_lsn`: Shows the latest WAL location replayed.
- `lag_bytes`: Calculates the difference between sent and replayed WAL locations.

Expected replication state:

streaming

6.4 Test replication

On the primary server, execute:

```
CREATE TABLE replication_test (
    id SERIAL PRIMARY KEY,
    message VARCHAR(100)
);
```

Insert a record:

```
INSERT INTO replication_test (message)
VALUES ('Streaming replication is working');
```

6.5 Verify the replicated data

Connect to the standby:

```
sudo -u postgres psql -p 5433 -d bootcamp
```

Execute:

```
SELECT * FROM replication_test;
```

Expected result:

```
id | message
---+--------------------------------
1  | Streaming replication is working
```

6.6 Check standby health

```
SELECT
    pg_is_in_recovery(),
    pg_last_wal_receive_lsn(),
    pg_last_wal_replay_lsn(),
    pg_last_xact_replay_timestamp();
```

The standby should report `true` for recovery mode and show WAL receive and replay positions.

---

7. PostgreSQL Terminal Commands Reference

Command	Purpose

`sudo -u postgres psql`	Open PostgreSQL terminal
`\l`	List databases
`\c bootcamp`	Connect to bootcamp
`\dt`	List database tables
`\d students`	Display table structure
`\du`	List database roles
`\conninfo`	Show connection information
`SELECT version();`	Display PostgreSQL version
`\q`	Exit PostgreSQL

---

8. Assignment Verification Checklist

- Created a logical backup using pg_dump.
- Inspected the backup using pg_restore.
- Restored the backup into bootcamp_check.
- Enabled WAL archiving.
- Verified WAL archiving statistics.
- Created a physical base backup.
- Simulated accidental data deletion.
- Configured point-in-time recovery.
- Verified recovered student records.
- Created the replicator role.
- Configured PostgreSQL authentication.
- Built a streaming standby.
- Verified standby recovery mode.
- Monitored replication health.
- Tested replication by inserting data on the primary and reading it on the standby.

---

9. Technologies and Tools Used

- PostgreSQL
- Linux / Ubuntu Terminal
- SQL
- pg_dump
- pg_restore

- pg_basebackup
- WAL Archiving
- Point-in-Time Recovery (PITR)
- Streaming Replication

---

10. Conclusion

This project demonstrates practical PostgreSQL database administration skills, including logical backups, physical backups, WAL archiving, point-in-time recovery, and streaming replication.

The exercises demonstrate how database administrators can protect important information, recover from accidental data loss, and maintain a standby database for improved availability and data protection.

Project Status: Complete upon successful execution and verification of all five steps.